반도체장비데이터관리 09

SemiconDB 확장하기 (2)

김 민 진 | muea.mj@gmail.com

샘플 데이터 재구성

현재 상태

- 의미적으로 맞지 않는 데이터가 많음
  - 직원의 소속된 부서와 관련 없는 장비 사용기록
  - 장비의 상태와 유지보수 기록이 연결되지 않음

→ 앞으로의 실습은 *데이터의 맥락이 중요*한 단계임.

현재 상태

UsageLog

log_id	user_id	equipment_id	use_date	issue_report
1	1	101	2024-03-01	챔버 압력 이상
2	2	103	2024-03-02	NULL
3	3	102	2024-03-05	연마 균일도 문제
4	1	104	2024-03-07	NULL
5	4	101	2024-03-08	이물 감지 센서 오작동
6	5	105	2024-03-10	NULL
7	3	103	2024-03-11	NULL
8	2	105	2024-03-12	예열 지연
9	1	102	2024-03-13	NULL
10	4	103	2024-03-14	패턴 비정상
11	5	101	2024-03-15	NULL
12	6	106	2024-03-18	초기 테스트 완료

현재 상태

Equipment

equipment_id	model_name	install_date	status
101	ETCH-A100	2021-05-10	active
102	CMP-X200	2023-11-23	maintenance
103	LITHO-Z300	2022-07-15	active
104	CVD-B500	2019-03-12	retired
105	DIFF-Y700	2023-01-20	active
106	CVD-C900	2024-03-20	maintenance
107	IMPLANT-Q400	2024-03-25	active

MaintenanceLog

maintenance_id	equipment_id	maintenance_date	engineer_name	maintenance_type	details
1	101	2024-03-03	최정우	정기점검	챔버 내부 청소 및 압력 점검
2	102	2024-03-06	박수현	수리	연마 균일도 문제로 부품 교체
3	104	2024-03-09	이도현	정기점검	노후 부품 교체 여부 검토
4	106	2024-03-21	김서윤	성능점검	신규 장비 설치 후 초기 테스트

현재 상태

EquipmentUser

user_id	name	contact	dept_id
1	김지훈	010-1234-5678	2
2	박서윤	010-2345-6789	1
3	이준호	010-3456-7890	2
4	김민아	010-4567-8901	4
5	정예린	010-5678-9012	3
6	김지후	010-6789-1111	5
7	한소라	010-7890-2222	4

Department

dept_id	dept_name	manager_name	office_location
1	제조팀	김부장	A동 2층
2	품질팀	이과장	B동 1층
3	연구팀	박차장	A동 1층
4	개발팀	최팀장	연구동 3층
5	생산기술팀	송팀장	연구동 3층

부서 구성

- 개선 사항: 실질적으로 장비를 사용할 법한 부서로 구성

- **공정운영팀**
  - 실제 장비 사용의 중심
  - ETCH, CVD, LITHO, CMP 같은 공정 장비 사용
- **품질분석팀**
  - 검사, 측정, 결과 확인 중심
  - 공정 장비를 직접 많이 쓰지는 않지만, 일부 확인용 사용 기록은 가능하게 구성
- **유지보수팀**
  - 장비 수리, 정기점검 담당
  - MaintenanceLog의 engineer_name에 등장하는 사람들

장비 상태

- 개선 사항: 상태를 다음처럼 나누고 유지보수 이력과 맞물리게 함

상태	의미	유지보수 이력과의 맞물림
active	정상 사용 가능	정기점검 이력이 있거나, 과거 수리 후 정상 운영 중
maintenance	현재 점검/수리 중	최근에 수리/점검 기록 있음
inactive	고장 또는 사용 중지 상태	최근 고장수리 이력이 있지만 아직 복구 전

컬럼 이름 축약

- 개선 사항: 컬럼 이름을 단축하여 코딩 편의성과 가독성을 향상시키자.

개선된 컬럼 이름 구성

```
Department(dept_id, dept_name, mgr_name, office)

EquipmentUser(user_id, name, dept_id, contact)

Equipment(eq_id, model, install_dt, status, status_dt)

UsageLog(log_id, user_id, eq_id, use_dt, issue)

MaintenanceLog(maint_id, eq_id, maint_dt, eng_name, maint_type, detail)
```

※ 외래키 컬럼의 이름을 변경할 때는 외래키 해제 → 컬럼명 변경 → 외래키 다시 설정 의 과정을 거쳐야 한다.

컬럼 이름 축약

- 축약 규칙

기존	축약	의미
id	id	그대로 유지
name	name	그대로 유지
department	dept	표준 축약
equipment	eq	자주 쓰는 축약
maintenance	maint	일반적 축약
date	dt	날짜
issue_report	issue	문제 기록
details	detail	단수형

컬럼 이름 축약

- 컬럼 이름 바꾸는 방법: ALTER TABLE에서 CHANGE를 사용한다.

기본 구조

```
ALTER TABLE [대상 테이블]
CHANGE [기존 컬럼명] [새로운 컬럼명] [데이터 타입];
```

컬럼 이름 변경 예시 코드

```
ALTER TABLE Department
CHANGE manager_name mgr_name VARCHAR(50),
CHANGE office_location office VARCHAR(50);
```

※ 컬럼 이름을 바꾸면, 지금까지의 쿼리를 전부 수정해야하므로, 알아만 두고 실제로 수정하지는 않겠습니다.

재구성 이후 데이터

UsageLog

log_id	user_id	equipment_id	use_date	issue_report
1	1	101	2024-03-04	NULL
2	2	102	2024-03-05	증착 압력 불안정
3	3	103	2024-03-05	NULL
4	1	101	2024-03-06	식각 균일도 편차 확인 필요
5	2	103	2024-03-07	NULL
6	4	105	2024-03-07	검사 결과 일부 이상 패턴 발견
7	5	105	2024-03-08	NULL
8	3	101	2024-03-08	NULL
9	1	103	2024-03-11	노광 정렬 오차 의심
10	4	105	2024-03-11	재검사 진행
11	2	101	2024-03-12	NULL

Equipment

equipment_id	model_name	install_date	status
101	ETCH-A100	2022-03-15	active
102	CVD-B200	2021-11-20	maintenance
103	LITHO-C300	2020-07-05	active
104	CMP-D400	2022-09-12	inactive
105	INSPECT-E500	2023-01-18	active

재구성 이후 데이터

MaintenanceLog

maintenance_id	equipment_id	maintenance_date	engineer_name	maintenance_type	details
1	105	2024-03-01	윤가은	정기점검	검사 센서 및 카메라 모듈 상태 점검
2	101	2024-03-02	박정민	정기점검	식각 챔버 내부 청소 및 센서 점검
3	103	2024-03-03	윤가은	정기점검	노광 정렬 장치 캘리브레이션 수행
4	102	2024-03-06	오지훈	긴급수리	압력 제어 밸브 이상으로 장비 점검 시작
5	101	2024-03-07	박정민	점검	식각 균일도 편차 원인 확인 및 공정 조건 ...
6	105	2024-03-08	윤가은	점검	이상 패턴 재검토 및 검사 모듈 보정
7	104	2024-03-09	박정민	고장수리	연마 패드 구동 이상으로 핵심 부품 교체
8	102	2024-03-10	오지훈	정기점검	수리 후 재가동 전 안전 점검 진행
9	103	2024-03-12	윤가은	점검	노광 정렬 오차 재점검 및 보정값 조정

재구성 이후 데이터

EquipmentUser

user_id	name	contact	dept_id
1	김현우	010-1111-1111	1
2	최민석	010-1111-2222	1
3	정다은	010-1111-3333	1
4	이서연	010-2222-1111	2
5	한지훈	010-2222-2222	2
6	박정민	010-3333-1111	3
7	오지훈	010-3333-2222	3
8	윤가은	010-3333-3333	3

Department

dept_id	dept_name	manager_name	office_location
1	공정운영팀	김현우	A동 1층
2	품질분석팀	이서연	B동 2층
3	유지보수팀	박정민	장비동 1층

실습 문제

- 데이터의 맥락이 왜 중요한지 문제를 통해 확인해보자.
- 각 문제의 **상황을 읽고, 필요한 정보를 정의한 뒤 쿼리로 확인하자.**
- 정답 쿼리는 한 가지가 아닐 수 있다.

실습 문제

1. 공정 중 이상이 보고된 장비가 이후 유지보수로 이어졌는지 확인해보자.
   (힌트: 이상 보고 기록과 유지 보수 기록을 연결)

2. 현재 maintenance 상태인 장비는 왜 지금 점검 중인지 확인해보자.
   (힌트: 최근 유지보수 이력과 연결해서 해석)

3. 공정운영팀은 어떤 장비를 사용하는가? 품질분석팀은 왜 특정 장비만 사용하는가?
   (힌트: 부서와 장비 사용 패턴 비교)

이 정보를 얻으려면 어떻게 해야할까?

1. 정보의 주체 찾기 (누구의 정보?)
   → 어느 테이블을 봐야 하나?
2. "어떤" 주체를 찾아야 하는가
   → 조건인가, 그룹인가, 집계인가, JOIN인가?
3. 주체의 무엇(정보)을 출력해야하는가?

실습 문제

4. 유지보수팀은 실제로 얼마나 작업을 수행하고 있는가?
   특정 엔지니어에게 업무가 집중되어 있는가?
   (힌트: 엔지니어별 작업 건수 비교)

5. 유지보수가 많은 장비는 문제 많은 장비일까, 아니면 관리가 잘 되고 있는 장비일까?
   (힌트: 횟수 + 유지보수 유형 함께 해석)

6. 사용되지 않는 장비가 있는지 확인해보자.
   이런 장비는 어떤 상태이며, 왜 사용되지 않았을까?
   (힌트: 사용기록 + 장비 상태 연결)

실습 문제

7. 장비를 사용하는 사람과 유지 보수하는 사람은 같은 부서일까?
   데이터로 확인해보자.
   (힌트: EquipmentUser ↔ Department ↔ MaintenanceLog 연결 필요)

8. 하나의 장비에서 일어난 일을 시간 순으로 정리해보자.
   (힌트: 사용 기록과 유지보수 기록을 함께 연결)

핵심 포인트

1. 데이터는 이야기다

- UsageLog = “사용 중 일어난 일”
- MaintenanceLog = “문제 해결 과정”

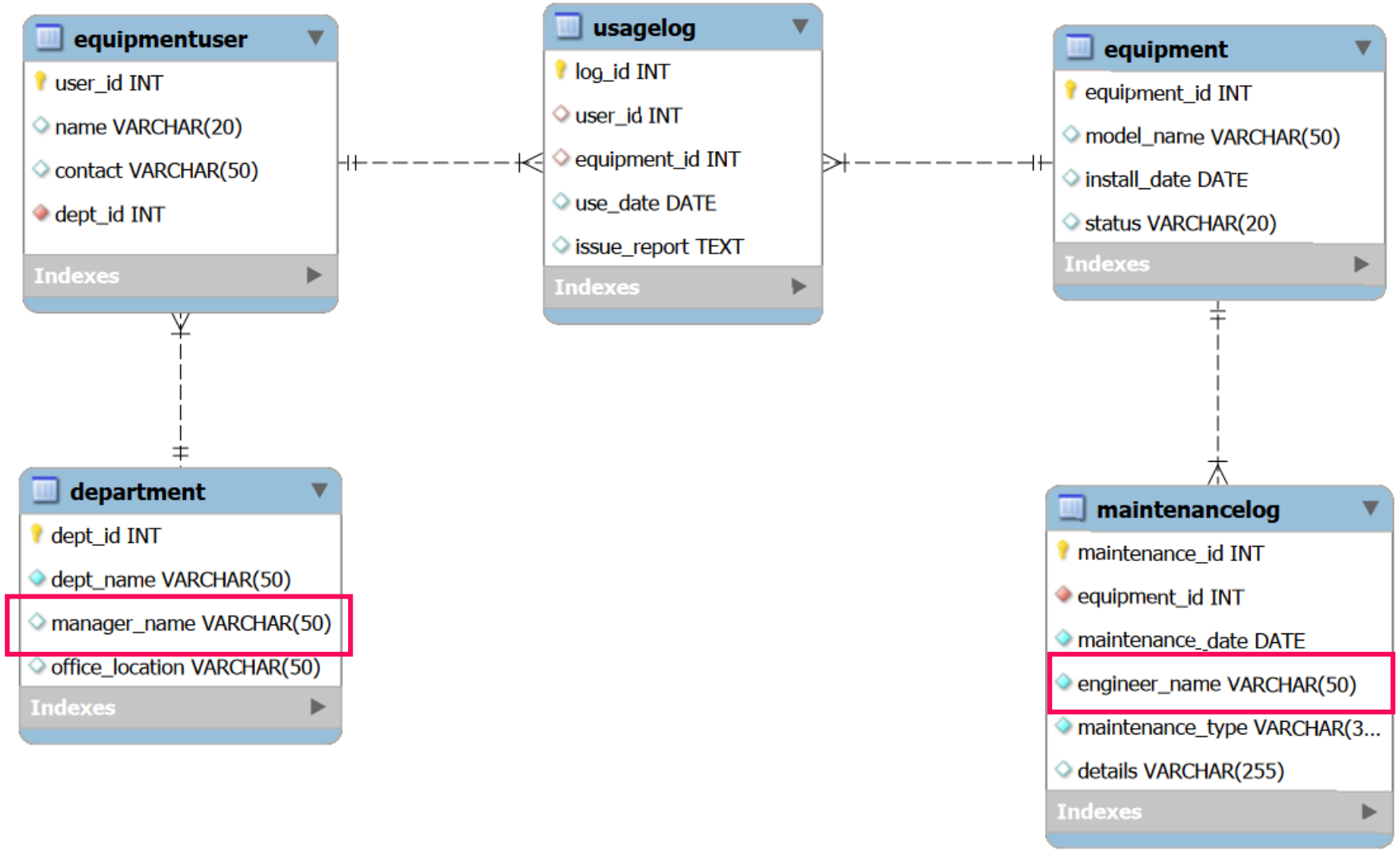
→ 둘을 연결하면 **스토리**가 된다

2. 구조가 맞아야 해석이 가능하다

- 데이터가 맞지 않으면, 쿼리는 돌아가도 해석이 틀린다.
- SQL은 데이터를 조회하는 기술이 아니라, **상황을 해석하는 도구**다.

3) Employee

구조 재검토


equipmentuser
user_id INT
name VARCHAR(20)
contact VARCHAR(50)
dept_id INT
Indexes
usagelog
log_id INT
user_id INT
equipment_id INT
use_date DATE
issue_report TEXT
Indexes
equipment
equipment_id INT
model_name VARCHAR(50)
install_date DATE
status VARCHAR(20)
Indexes
department
dept_id INT
dept_name VARCHAR(50)
manager_name VARCHAR(50)
office_location VARCHAR(50)
Indexes
maintenancelog
maintenance_id INT
equipment_id INT
maintenance_date DATE
engineer_name VARCHAR(50)
maintenance_type VARCHAR(3...
details VARCHAR(255)
Indexes

구조 재검토

- 장비 사용 로그를 중심으로 한 간단한 구조에서는 EquipmentUser가 장비 사용자만을 의미해도 괜찮았음

- 구조가 확대되며 장비 사용자 외에도 사람을 표현하는 구조가 필요해짐
  - Department.**manager_name**
  - MaintenanceLog.**engineer_name**

  → **이대로 두기 구조상 좋지 않지만, 이들을 위한 테이블을 따로 만들기에도 좋지 않다.**

  → **각 직원의 역할마다 테이블을 하나씩 만들 순 없다.**

*→ EquipmentUser를 **직원(Employee)을 나타내는 테이블**로 확대할 필요가 있음*

작업할 내용

- EquipmentUser → Employee로 의미 확장

- 문자열 컬럼을 참조형(FK)으로 전환
  - engineer_name → engineer_id
  - manager_name → manager_id

EquipmentUser → Employee로 이름 변경

- 목적: "장비 사용자" → "조직의 직원"으로 의미를 확장.

```sql
-- ============================================================
-- 1. EquipmentUser -> Employee 테이블명 변경
-- ============================================================
RENAME TABLE EquipmentUser TO Employee;


-- ============================================================
-- 2. Employee의 user_id -> employee_id로 컬럼명 변경
-- ============================================================
ALTER TABLE Employee
CHANGE user_id employee_id INT;


-- ============================================================
-- 3. UsageLog의 user_id -> employee_id로 컬럼명 변경
-- ============================================================
ALTER TABLE UsageLog
CHANGE user_id employee_id INT;
```

① 확장된 의미를 반영하여 테이블 이름을 변경한다

② 컬럼명도 함께 정리한다.

③ EquipmentUser를 참조하고 있는 테이블(UsageLog)의 컬럼명도 정리한다.

MaintenanceLog: engineer_name → engineer_id

```sql
-- ============================================================
-- 4. MaintenanceLog: engineer_name -> engineer_id
-- ============================================================

-- 4-1. 새 컬럼 추가
ALTER TABLE MaintenanceLog
ADD engineer_id INT;

-- 4-2. 이름 기준으로 engineer_id 매핑
UPDATE MaintenanceLog m
JOIN Employee e
  ON m.engineer_name = e.name
SET m.engineer_id = e.employee_id;

-- 4-3. 매핑 결과 확인
SELECT maintenance_id, engineer_name, engineer_id
FROM MaintenanceLog;
```

```sql
-- 4-4. 매핑되지 않은 데이터 확인
SELECT *
FROM MaintenanceLog
WHERE engineer_id IS NULL;

-- 4-5. 모든 행이 정상적으로 매핑된 뒤 NOT NULL 설정
ALTER TABLE MaintenanceLog
MODIFY engineer_id INT NOT NULL;

-- 4-6. 외래키 추가
ALTER TABLE MaintenanceLog
ADD CONSTRAINT fk_maintenance_engineer
FOREIGN KEY (engineer_id)
REFERENCES Employee(employee_id);

-- 4-7. 기존 engineer_name 컬럼 삭제
ALTER TABLE MaintenanceLog
DROP COLUMN engineer_name;
```

Department: manager_name → manager_id

```
-- ============================================================
-- 5. Department: manager_name -> manager_id
-- ============================================================

-- 5-1. 새 컬럼 추가

-- 5-2. 이름 기준으로 manager_id 매핑

-- 5-3. 매핑 결과 확인

-- 5-4. 매핑되지 않은 데이터 확인
```

```
-- 5-5. manager_id가 모두 채워졌다면 외래키 추가

-- 5-6. 기존 manager_name 컬럼 삭제
```

끝!